UNIVERSIDAD POLITECNICA SALESIANA - SEDE QUITO

ASIGNATURA: Sistemas Distribuidos

DOCENTE: Julio Proaño Ricardo Orellana

CARRERA: Computación

GRUPO: 1

INTEGRANTES:

Jairo Alejandro Ojeda Herrera

Adrián Fernando Barriga Castillo

FECHA: 06-11-2026

Tabla de contenido

1. Resumen ejecutivo 3
2. Objetivos 3
2.1 Objetivo general 3
2.2 Objetivos específicos 3
3. Herramientas y tecnologías 3
4. Arquitectura y funcionamiento 4
4.1 Flujo de registro 4
4.2 Flujo de inicio de sesión y segundo factor 4
5. Descripción de los archivos entregados 5
6. Estructura de la base de datos 5
6.1 Tabla usuarios 5
6.2 Tabla codigos_mfa 5
7. Controles de seguridad identificados 5
8. Evidencias disponibles 6
Evidencia 1. Creación del bot en Telegram 6
Evidencias que faltan para el informe final ..... **¡Error! Marcador no definido.**
9. Observaciones técnicas y aspectos por verificar 8
10. Pruebas funcionales propuestas 9
11. Resultados y alcance de la revisión 9
12. Conclusiones 10
13. Recomendaciones 10
14. Declaración de participación 10

Link de GitHub: https://github.com/Alejandro120325/Pyro4_Telegram.git

1. Resumen ejecutivo

El proyecto implementa un mecanismo de autenticación distribuida basado en una arquitectura cliente-servidor. El cliente, desarrollado en Python, permite registrar usuarios e iniciar sesión. La comunicación con el servidor se realiza mediante Pyro4, que expone métodos remotos para el registro, la validación de credenciales y la verificación de un segundo factor.

Como segundo factor de autenticación (MFA), el servidor genera un código numérico de seis dígitos y lo envía al chat de Telegram asociado al usuario. El código tiene una vigencia de cinco minutos y el sistema limita su validación a tres intentos. La información de usuarios y códigos temporales se almacena en SQLite.

Este informe se elaboró a partir de la revisión del archivo comprimido Pyro4_Telegram.zip y de la única captura adicional recibida. Por tanto, documenta la estructura y el comportamiento identificables en el código; no afirma pruebas de ejecución que no estén respaldadas por evidencias.

2. Objetivos

2.1 Objetivo general

Documentar la implementación de un sistema de autenticación distribuida que integra Pyro4, Python, SQLite y Telegram para aplicar autenticación de dos factores.

2.2 Objetivos específicos

- Describir la arquitectura y la comunicación entre el cliente y el servidor.
- Explicar el proceso de registro y almacenamiento seguro de credenciales.
- Detallar el inicio de sesión y la entrega del código MFA mediante Telegram.
- Identificar los controles de vigencia, intentos y validación del código.
- Registrar las evidencias disponibles y señalar los aspectos técnicos que deben verificarse antes de una demostración final.

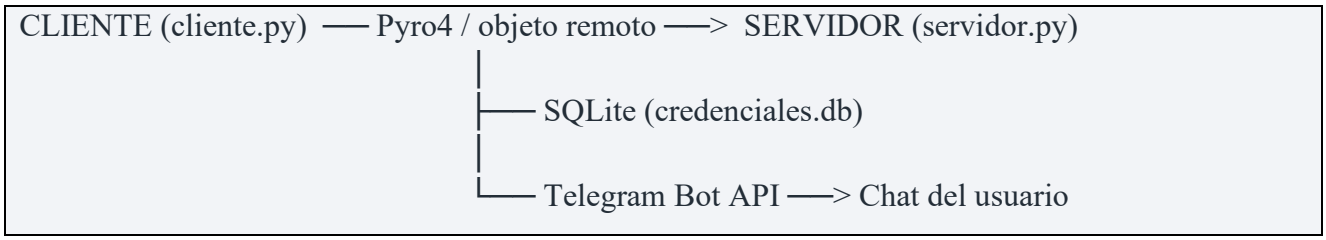
3. Herramientas y tecnologías

Tecnología	Uso dentro del proyecto	Evidencia en archivos
Python	Lenguaje de programación del cliente y servidor.	cliente.py, servidor.py, obtener_chat_id.py
Pyro4	Invocación de métodos remotos y publicación del objeto de autenticación.	Importación, Proxy, Daemon y decoradores en cliente.py y servidor.py
SQLite	Persistencia local de usuarios y códigos MFA temporales.	credenciales.db y creación de tablas
Telegram Bot API	Envío de notificaciones y códigos de verificación.	requests y endpoint sendMessage

Requests	Realización de solicitudes HTTP hacia Telegram.	servidor.py y obtener_chat_id.py
PBKDF2-HMAC-SHA256	Derivación de hash de contraseñas con sal.	función hashear() en servidor.py

4. Arquitectura y funcionamiento

La solución se organiza en dos componentes principales: un cliente de consola y un servidor de autenticación. El cliente solicita los datos al usuario y utiliza un objeto Proxy de Pyro4 para invocar operaciones remotas. El servidor procesa las solicitudes, consulta o actualiza la base SQLite y se comunica con Telegram cuando debe enviar una notificación o un código de verificación.



4.1 Flujo de registro

- El usuario selecciona la opción Registrarse en el cliente.
- El cliente solicita nombre de usuario, contraseña y chat_id de Telegram.
- El servidor verifica que usuario y contraseña no estén vacíos.
- Se genera una sal aleatoria y se deriva el hash de la contraseña mediante PBKDF2-HMAC-SHA256 con 200 000 iteraciones.
- Se almacena usuario, hash, sal y chat_id en la tabla usuarios.
- El servidor intenta enviar una notificación de registro a Telegram. Si el envío falla, elimina el registro recién creado y devuelve un mensaje de error.

4.2 Flujo de inicio de sesión y segundo factor

- El cliente solicita usuario y contraseña; getpass evita mostrar la contraseña mientras se escribe en la consola.
- El servidor recupera el hash, la sal y el chat_id del usuario.
- Se vuelve a calcular el hash con la contraseña ingresada y se compara con secrets.compare_digest.
- Si las credenciales son correctas, se genera un código aleatorio de seis dígitos.
- El código se guarda con una expiración de 300 segundos y se envía al chat de Telegram asociado.
- El usuario ingresa el código en el cliente. Si es correcto, el servidor elimina el código temporal y autoriza el acceso.
- Si el código es incorrecto, se incrementa el contador. Al alcanzar tres intentos o superar el tiempo de vigencia, el código se elimina.

5. Descripción de los archivos entregados

cliente.py — Interfaz de consola. Presenta las opciones de registro, inicio de sesión y salida; solicita credenciales y realiza llamadas remotas al servidor.

servidor.py — Contiene la lógica de autenticación, creación de tablas, generación y validación de códigos, hash de contraseñas e integración con Telegram.

obtener_chat_id.py — Consulta getUpdates de Telegram para obtener el identificador de chat de los mensajes recibidos por el bot.

credenciales.db — Base de datos SQLite. En el archivo revisado se identificaron las tablas usuarios y codigos_mfa.

requirements.txt — Declara las dependencias Pyro4 y requests.

6. Estructura de la base de datos

6.1 Tabla usuarios

La tabla usuarios utiliza usuario como clave primaria y contiene hash, sal y chat_id. No se observa un campo para almacenar la contraseña en texto plano. La contraseña se transforma antes de persistirse.

6.2 Tabla codigos_mfa

La tabla codigos_mfa almacena un registro temporal por usuario: código, instante de expiración e intentos realizados. El usuario es clave primaria, por lo que el nuevo código reemplaza al anterior mediante INSERT OR REPLACE.

Tabla	Campos	Propósito
usuarios	usuario, hash, sal, chat_id	Datos persistentes de la cuenta y destino de Telegram.
codigos_mfa	usuario, codigo, expira, intentos	Código temporal y control de validaciones.

7. Controles de seguridad identificados

- Hash de contraseñas con PBKDF2-HMAC-SHA256 y 200 000 iteraciones.
- Sal aleatoria de 16 bytes generada mediante secrets.token_hex(16).
- Comparación de hashes y códigos mediante secrets.compare_digest.
- Generación criptográficamente segura del código MFA de seis dígitos.
- Vigencia del código limitada a cinco minutos.
- Máximo de tres intentos de validación del código.
- Uso de variable de entorno TELEGRAM_TOKEN para no escribir el token directamente en el código fuente.
- Uso de getpass para ocultar la contraseña en la consola.
- Tiempo máximo de espera de diez segundos en el envío HTTP a Telegram.

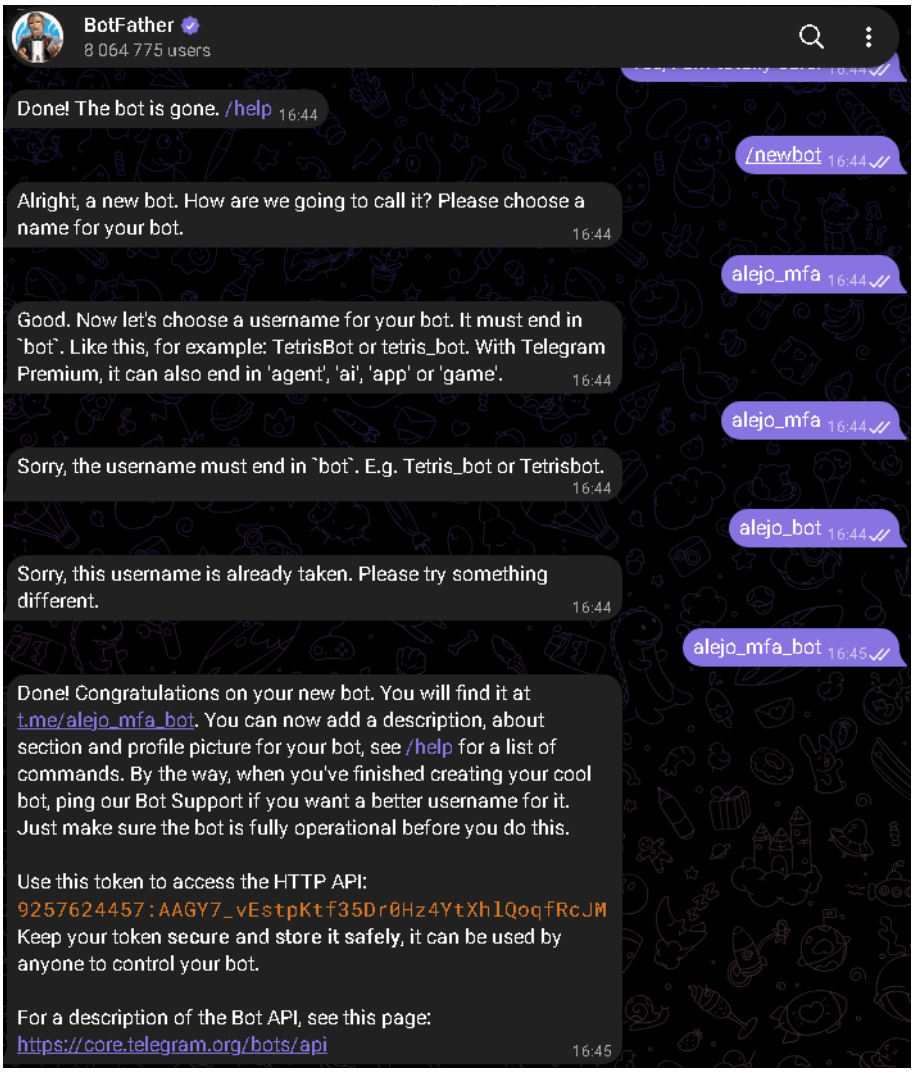
8. Evidencias disponibles

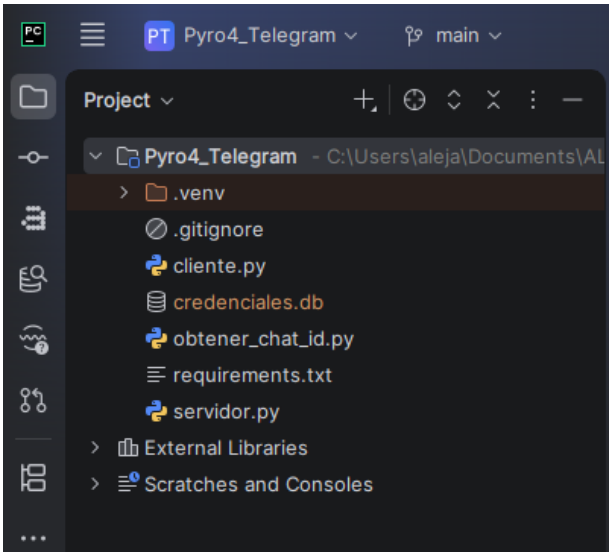
En los archivos recibidos se encontró una captura relacionada con la creación del bot en Telegram. La imagen contiene información sensible de acceso; por seguridad, no se reproduce el token ni se incorpora la captura original sin censura. La evidencia se registra aquí mediante una descripción textual.

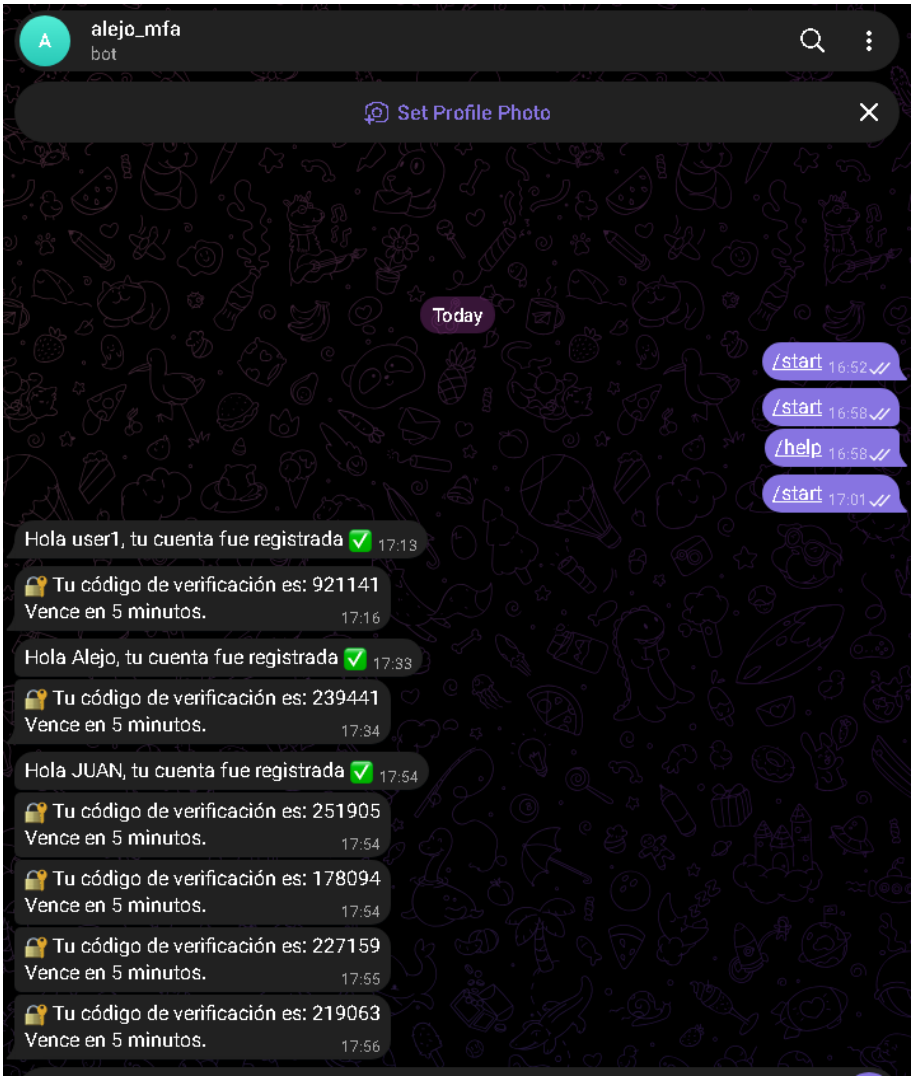
Evidencia 1. Creación del bot en Telegram

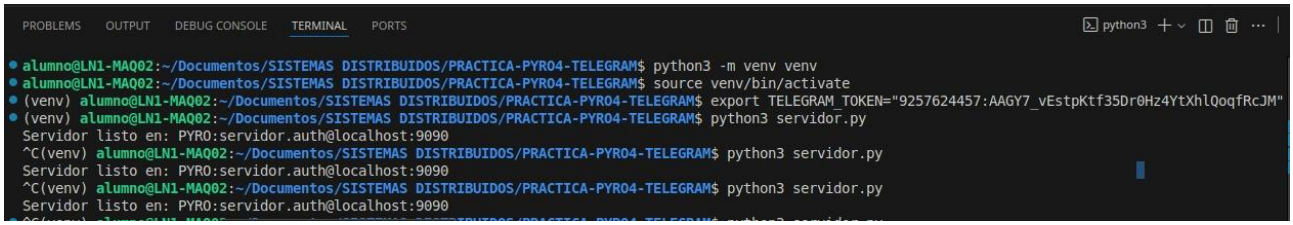
La captura recibida corresponde al mensaje de confirmación de BotFather sobre la creación del bot y muestra el enlace de acceso al bot. También contiene una credencial privada, por lo que debe mantenerse fuera del informe y ser revocada si continúa activa.

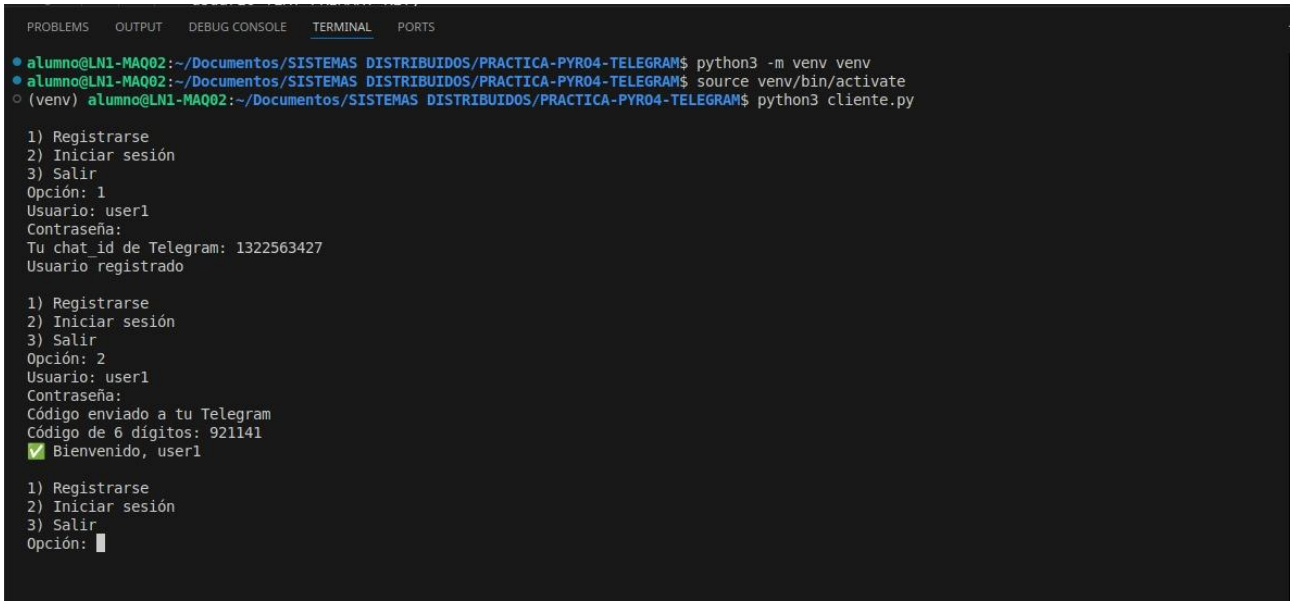
Además de la captura, el código fuente constituye evidencia documental de los componentes cliente y servidor. Se revisaron cliente.py, servidor.py, obtener_chat_id.py, requirements.txt y la estructura de credenciales.db.




Pyro4_Telegram
main
Project
Pyro4_Telegram - C:\Users\aleja\Documents\AL
.venv
.gitignore
cliente.py
credenciales.db
obtener_chat_id.py
requirements.txt
servidor.py
External Libraries
Scratches and Consoles


alejo_mfa
bot
Set Profile Photo
Today
/start 16:52
/start 16:58
/help 16:58
/start 17:01
Hola user1, tu cuenta fue registrada ✅ 17:13
Tu código de verificación es: 921141
Vence en 5 minutos. 17:16
Hola Alejo, tu cuenta fue registrada ✅ 17:33
Tu código de verificación es: 239441
Vence en 5 minutos. 17:34
Hola JUAN, tu cuenta fue registrada ✅ 17:54
Tu código de verificación es: 251905
Vence en 5 minutos. 17:54
Tu código de verificación es: 178094
Vence en 5 minutos. 17:54
Tu código de verificación es: 227159
Vence en 5 minutos. 17:55
Tu código de verificación es: 219063
Vence en 5 minutos. 17:56





9. Observaciones técnicas y aspectos por verificar

Durante la revisión estática del código se identificó un punto importante de configuración de red: el servidor crea el daemon Pyro4 con host="localhost", mientras que el cliente utiliza una URI con la dirección 172.17.44.90 y puerto 9090. Si el cliente se ejecuta desde otro equipo, un servicio enlazado únicamente a localhost no será accesible desde la red. Para una comunicación entre equipos debe configurarse el daemon para escuchar en una interfaz alcanzable (por ejemplo, la IP privada del servidor o 0.0.0.0, según el entorno), revisar el firewall y anunciar una URI accesible. Esta configuración debe probarse antes de afirmar que la comunicación remota está operativa.

También se recomienda revisar los siguientes puntos antes de la entrega:

- Revocar el token que apareció en la captura compartida y generar uno nuevo; guardarlo únicamente como variable de entorno TELEGRAM_TOKEN.
- Configurar Pyro4 con mecanismos de seguridad y evitar exponer el puerto 9090 directamente a redes no confiables.
- Considerar cifrado del canal de comunicación, ya que el código revisado no configura TLS para Pyro4.
- Evitar que el código MFA permanezca almacenado en texto plano en SQLite; evaluar almacenar un hash del código.

- Agregar límites de intentos y demoras para los intentos de contraseña, no solamente para el código MFA.
- Validar que el usuario haya iniciado conversación con el bot mediante /start y que el chat_id corresponda al usuario esperado.
- Comprobar el comportamiento cuando Telegram no está disponible y cuando ocurren errores de conexión Pyro4.

Estas observaciones no invalidan la estructura implementada; representan verificaciones y mejoras recomendadas para fortalecer la seguridad y garantizar el funcionamiento entre equipos.

10. Pruebas funcionales propuestas

Prueba	Acción	Resultado esperado	Estado documental
Registro válido	Ingresar usuario, contraseña y chat_id correctos.	Crear usuario y enviar confirmación por Telegram.	Implementación identificada; falta captura de ejecución.
Usuario duplicado	Registrar un usuario ya existente.	Informar que el usuario ya existe.	Control presente en código.
Credenciales válidas	Ingresar usuario y contraseña correctos.	Enviar código de seis dígitos a Telegram.	Implementación identificada; falta captura de ejecución.
Contraseña incorrecta	Ingresar una contraseña no válida.	Rechazar el acceso.	Control presente en código.
Código correcto	Ingresar el MFA recibido dentro de cinco minutos.	Validar el acceso y eliminar el código temporal.	Implementación identificada; falta captura de ejecución.
Código incorrecto	Ingresar un código diferente.	Incrementar intentos y permitir reintento hasta el límite.	Control presente en código.
Código expirado	Esperar más de cinco minutos.	Rechazar el código y solicitar nuevo inicio de sesión.	Control presente en código.
Tres intentos fallidos	Ingresar tres códigos incorrectos.	Eliminar el código temporal y bloquear nuevos intentos para ese código.	Control presente en código.

11. Resultados y alcance de la revisión

La revisión de los archivos permite constatar la existencia de una solución organizada en cliente, servidor, persistencia SQLite e integración con Telegram. Se identificaron los métodos remotos

registrar(), iniciar_sesion() y verificar_codigo(), así como los controles de hash de contraseñas, generación de códigos temporales, expiración y límite de intentos.

No se dispone todavía de capturas de ejecución del cliente y del servidor. Por ello, los resultados descritos corresponden a la inspección del código y de la base de datos entregados, y no deben interpretarse como una certificación independiente de pruebas en vivo.

12. Conclusiones

- Se implementó en el código una arquitectura cliente-servidor mediante Pyro4 para separar la interfaz de usuario de la lógica de autenticación.
- La integración con Telegram permite plantear un segundo factor de autenticación mediante códigos temporales de seis dígitos.
- El almacenamiento de contraseñas utiliza hash con sal y PBKDF2-HMAC-SHA256, evitando conservar contraseñas en texto plano.
- SQLite mantiene separadas la información persistente de usuarios y la información temporal de los códigos MFA.
- Antes de la demostración final es indispensable corregir o verificar el enlace de red del daemon Pyro4, porque localhost no permite por sí solo conexiones desde otros equipos.
- Para completar la constancia visual del trabajo se requieren capturas reales del servidor, cliente, mensajes de Telegram y resultados de las pruebas.

13. Recomendaciones

- Realizar una prueba de conectividad entre cliente y servidor desde los equipos que se utilizarán en la exposición.
- Tomar capturas legibles de cada etapa, ocultando contraseñas, tokens, códigos MFA y datos personales.
- Entregar el proyecto sin la carpeta .venv y sin archivos temporales; incluir requirements.txt para reproducir el entorno.
- Rotar inmediatamente cualquier token que haya sido expuesto en capturas o mensajes.
- Completar la matriz de pruebas con el resultado real observado y adjuntar la evidencia correspondiente.

14. Declaración de participación

El presente documento registra el trabajo técnico asociado al proyecto desarrollado en conjunto por Alejandro Ojeda y Adrián Castillo. La distribución específica de tareas individuales debe completarse de acuerdo con las actividades que cada integrante haya realizado efectivamente.